

Module 04

Dockerfile and Registry

DevOps end to end services & solutions

Agenda

- > Dockerfile – Basic commands
- > Dockerfile – Based on python:<os ver tag>
- > Dinner Suggestion example
- > Docker Build and Docker Run Image
- > Multi-Stage Build
- > Hands-On – Lab-04 – Dockerfile and Registry

What is a Dockerfile?



Blueprint

A script containing a set of instructions to automate the building of Docker images.



Setup

Defines everything needed to set up an app inside a container (OS, code, dependencies).



Automation

Contains all the commands a user could call on the command line to assemble an image.

Dockerfile – Basic Commands

- > **FROM:** Specifies the base image.
- > **WORKDIR:** Sets the working directory inside the container.
- > **COPY & ADD:** Copies files from host to container.
- > **RUN:** Executes commands during build time.
- > **CMD & ENTRYPOINT:** Defines the default command when running the container.
- > **EXPOSE:** Opens a port for the container.

```
FROM ubuntu:22.04
RUN apt-get update -y
RUN apt-get upgrade -y
RUN apt-get install -y curl
```

Hands-On – Lab 04: Dinner Suggestion

- > Create a app.py application (Python Flask).
- > Wrap it with a Dockerfile.
- > Build the image.
- > Run the image and access via browser.
- > Test via interactive mode.
- > Tag and Push to Docker Hub.



Dockerfile – Dinner Suggestion

*#Uses official Python image (version 3.9) ,slim = lightweight (smaller size,) This is your **starting point***

FROM python:3.9.slim

Set the working directory in the container

WORKDIR /myapp

#Copies everything from your project → container

COPY ..

Installs Python dependencies and Reads req.txt Installs flask packages, Prevents caching, its smaller image size

RUN pip install --no-cache-dir -r requirements.txt

Make the container's port 5000 available to the outside

EXPOSE 5000

ENV FLASK_APP=app.py *#Sets environment variable*

CMD ["flask", "run", "--host=0.0.0.0"] *#Default command when container starts*

Dockerfile another Example – Based on node:14 OS

```
# Use an official Node.js runtime as a parent image
FROM node:14

# Set the working directory in the container
WORKDIR /usr/src/app

# Copy package.json and package-lock.json to the working directory
COPY package*.json ./

# Install any needed dependencies
RUN npm install

# Copy the rest of the application code to the working directory
COPY . .

# Make the container's port 3000 available to the outside
EXPOSE 3000

# Define the command to run the application
CMD ["npm", "start"]
```

Docker Build and Run

Docker Build

Creates an image from a Dockerfile.
Packages your code to be shipped
anywhere.

```
docker build -t node-app .
```

Docker Run

Creates a running container from an
image. Use `-d` for background mode.

```
docker run -p 3000:3000 node-app  
# Or run in background  
docker run -d -p 3000:3000 node-app
```


Multi-Stage Build

Multi-stage builds allow you to drastically reduce the size of your final image by using multiple FROM instructions.

You can copy artifacts from one stage to another, leaving behind all the build tools and intermediate files.

```
1  # Stage - Development
2  FROM Dev-Image as dev
3  . . .
4  . . .
5
6  # Stage - Testing
7  FROM Test-Image as test
8  COPY --from=dev /src/app .
9  . . .
10 . . .
11
12 # Stage - Production
13 FROM Minimum-Image as prod
14 COPY --from=dev /src/app .
15 . . .
16 . . .
```

stage 0

stage 1

stage 2

Hands-On – Lab 04: Dinner Suggestion

- > Create a app.py application (Python Flask).
- > Wrap it with a Dockerfile.
- > Build the image.
- > Run the image and access via browser.
- > Test via interactive mode.
- > Tag and Push to Docker Hub.



Questions?

Thank you for participating.

Feel free to reach out:
<https://dinghy-e2e.com>

